Faculty of Electrical Engineering, Mathematics and Computer Science

DQsym: A MATLAB/Simulink library for power system simulation using dynamic phasors

CONTENTS

1 DQsym Library Folder Structure **4**
1.1 Top-Level Structure 4
1.2 `DQsym ToolBox` Folder (Core Library) 4
1.3 Features 5
1.4 Usage 6
1.5 Installation 6
1.6 Typical User Workflow 6
1.7 Result Verification 6
1.8 Development of New Test Cases 7
1.9 Test Case Folders 8
1.9.1 Common Contents of a Test Case Folder 8

2 Developed test cases **9**
2.1 Multiplication of harmonic-rich signals 9
2.1.1 Test Description 9
2.1.2 Model Implementation 9
2.1.3 Simulation and Results 10
2.2 A simple RLC circuit 10
2.2.1 Basic Test Description 10
2.2.2 Parameters 11
2.2.3 MATLAB Structure 11
2.2.4 Run the Test Case and Results 11
2.3 MMC Station Connected to an AC Grid 13
2.3.1 Basic Test Description 13
2.3.2 Parameters 14
2.3.3 MATLAB Structure 14
2.3.4 Run the Test Case and Results 14
2.4 Point-to-Point HVDC System with MMC 14
2.4.1 Basic Test Description 14
2.4.2 Parameters 17
2.4.3 MATLAB Structure 17
2.4.4 Run the Test Case and Results 17

1

DQSYM LIBRARY FOLDER STRUCTURE

This chapter describes the organisation and purpose of the `DQsym` library folder. The structure separates core library components from example and validation test cases, making the project easier to navigate and extend.

1.1 Top-Level Structure

- `DP-GitHUB/`
  - `DQsym ToolBox/`
  - `Multiplication/`
  - `RLC/`
  - `P2PHVDCMMc/`
  - `SingleMMc/`

1.2 `DQsym ToolBox` Folder (Core Library)

The `DQsym ToolBox` folder contains the core components of the library. It includes all reusable models, functions, and utilities required by the test cases, as shown in Fig. 1.2.1.

- All library blocks and utilities are located in this folder.
- The folder must be added to the MATLAB/Simulink path for simulations to run correctly.
- Users typically do not modify this folder unless extending the library.

The library includes source and grid interface blocks that provide voltage or current inputs in the *DQsym* dynamic phasor format, as defined in equation (1.2.1). The state-space block enables the implementation of dynamic models for converters, network elements, and control systems.

Mathematical operation blocks, including multiplication and integration, provide reusable and parameterised components for dynamic phasor modelling. These blocks support scalar,

vector, and matrix operations, including harmonic-coupled computations. Measurement and post-processing blocks convert internal *DQsym* signals into time-domain waveforms for visualisation and comparison with EMT simulations. Additional utility blocks support signal routing and integration with standard Simulink components.

$$\begin{bmatrix} \langle x_{dq,p} \rangle_k \\ \langle x_{dq,n} \rangle_k \\ \langle x_{dq,z} \rangle_k \end{bmatrix} = \begin{bmatrix} \mathbf{x}_d^p(k) + j\mathbf{x}_q^p(k) \\ \mathbf{x}_d^n(k) + j\mathbf{x}_q^n(k) \\ \mathbf{x}_d^z(k) + j\mathbf{x}_q^z(k) \end{bmatrix} \tag{1.2.1}$$

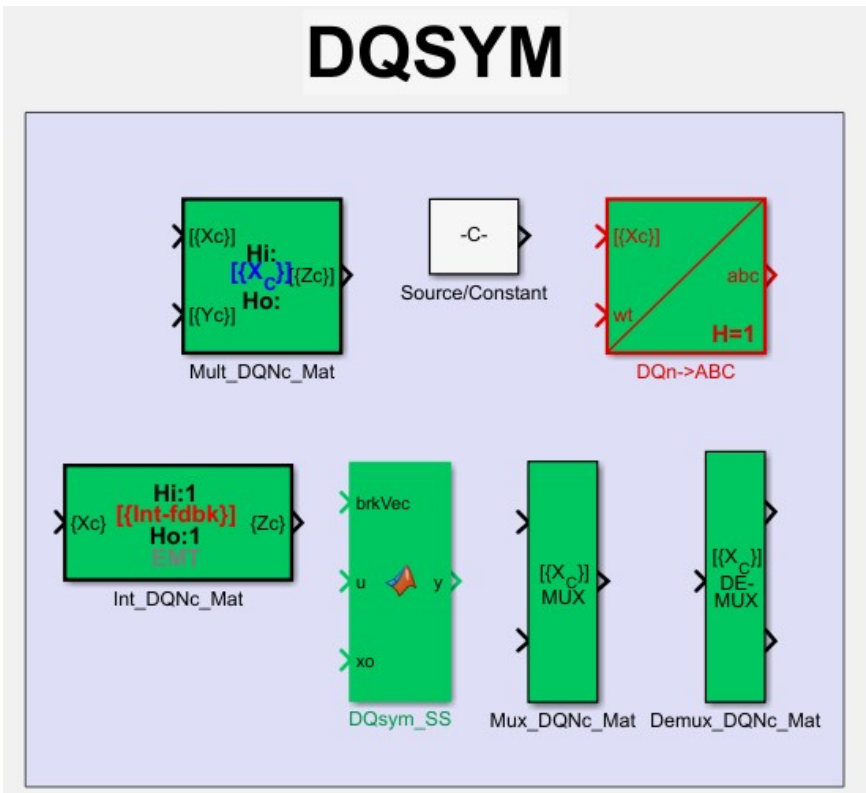


Figure 1.2.1: Overview of the *DQsym* library in MATLAB/Simulink.

1.3 Features

Version 1.0 includes:

- Masked blocks:
  - State-space block
  - Multiplication block
  - Integration block
  - Mux/Demux blocks
  - DQsym-to-ABC transformation block
- Access to internal block implementations through masked subsystems

- Modular Multilevel Converter (MMC) using an averaged state-space model
- A unified multi-harmonic state-space framework for modelling and analysis

1.4 Usage

- The library supports higher-order harmonic modelling using dynamic phasors. To capture these harmonics, the corresponding system states must be included in the model.
- Example models are provided using both SPS and `DQsym`. Users can enable or disable the SPS models for comparison.
- Scope and measurement blocks are available for monitoring outputs and internal states.

1.5 Installation

Follow these steps to install the `DQsym` library:

1. Download or clone the repository
2. Open MATLAB
3. Navigate to the `DQsym ToolBox` folder
4. Run the script `addlib.m`
5. Confirm adding the folder to the MATLAB path
6. The Simulink Library Browser will refresh automatically

MATLAB R2024a or later is recommended. Simulink, Simscape Electrical, the Signal Processing Toolbox, and the DSP System Toolbox are required for full functionality.

1.6 Typical User Workflow

1. Add the `DQsym ToolBox` and the desired test case folder to the MATLAB path
2. Navigate to the test case folder
3. Run the associated `.m` script
4. The script loads parameters and executes the Simulink model

1.7 Result Verification

At this stage, results are validated by comparison with reference MATLAB/Simulink models. Outputs are checked for consistency and overlap with EMT-based simulations.

1.8 Development of New Test Cases

Custom test cases can be developed by defining the system in discrete-time state-space form. The `DQsym` framework operates directly on system matrices, allowing consistent modelling of networks, converters, and control systems.

Standard components such as AC sources, transformers, and RLC elements can be implemented by deriving their state-space representations and connecting them using the provided blocks.

The framework supports averaged converter models. In Version 1.0, the MMC averaged model has been implemented and validated. Other converter models can be added using the same approach, but their implementation and validation are the responsibility of the user.

This approach provides flexibility but assumes familiarity with state-space modelling.

Sources in `DQsym` The `DQsym` library includes a source block for generating signals in dynamic phasor form. The block allows the user to define signals up to a selected harmonic order N.

A signal containing a DC component and a fundamental frequency component is represented as a $3 \times (1 + 1)$ matrix. Each additional harmonic order requires one extra column. In general, the signal is represented as a matrix of size $3 \times (1 + N)$.

Multiplication in `DQsym` The multiplication block requires at least two input signals. The masked block parameter defines the harmonic order of the output signal.

The output harmonic order must be set to at least twice the highest harmonic order present in the input signals. This ensures correct representation of harmonic coupling terms generated during multiplication.

Integration in `DQsym` The integration block requires at least one input signal. The masked block parameter defines the harmonic order of the output. The output harmonic order should be equal to the highest harmonic order of the input signal.

DQn to ABC Transformation in `DQsym` The DQn-to-ABC transformation block converts signals from the *DQsym* representation to the time-domain *abc* frame. The block requires at least one input signal. The harmonic order parameter must match the highest harmonic order present in the input signal to ensure accurate waveform reconstruction.

Mux and Demux in `DQsym` The Mux and Demux blocks are used to combine or separate *DQsym*-formatted signals. The masked block parameter defines the number of input or output signals. These blocks facilitate signal routing and integration within larger models.

State-Space and Input Definition in `DQsym` The system must be defined using state-space matrices (**A**, **B**, **C**, **D**). These matrices are implemented directly within the State-Space block. The system matrices do not need to be converted into the *DQsym* format. The internal algorithm of the block handles the required transformation.

The masked block parameters include:

- State-space matrices (**A**, **B**, **C**, **D**)
- Initial conditions
- Number of switching elements

- Switch type (0 or 1)
- On-state and off-state resistances
- Simulation time step
- System frequency

The block inputs include:

- Switch state vector (`brkvec`)
- Input vector (`u`)

All input signals (**u**) must be provided in the *DQsym* format. This is achieved using the dedicated source and interface blocks, ensuring compatibility with the internal signal structure and reference frames.

1.9 Test Case Folders

The folders `Multiplication`, `RLC`, `P2PHVDCMMc`, and `SingleMMc` contain validated test cases. These are implemented using both:

- The `DQsym` library
- The Simulink *Specialized Power Systems* (SPS) library

1.9.1 Common Contents of a Test Case Folder

- `slprj/` Automatically generated Simulink cache and build files. This folder can be deleted and regenerated.
- `*.slx` Simulink model files defining the system topology, including electrical networks and control systems.
- `*.m` MATLAB scripts used to run the test case. These scripts:
  - Load system parameters
  - Configure the simulation
  - Execute the corresponding Simulink model
- `*.mat` Data files containing parameters or precomputed variables.

2

DEVELOPED TEST CASES

2.1 Multiplication of harmonic-rich signals

This section presents a simple example that demonstrates the multiplication of harmonic-rich signals using the `DQsym` library. The example highlights the ability of the framework to capture harmonic interactions and validate the correctness of the implemented dynamic phasor operations.

$$
\begin{aligned}
\mathbf{Z_{DQsym}} &= \mathbf{X_{DQsym}} \times \mathbf{Y_{DQsym}} + 2 \times \mathbf{Y_{2_{DQsym}}} \\
&= \left(\begin{bmatrix} 0 & -0.88+0.40i & -0.01+0.00i & -0.12-0.12i & -0.34-0.13i & -0.89-0.09i \\ 0 & 0.04-0.69i & -0.11-0.65i & 0.47+0.87i & -0.66+0.34i & 0.40+0.58i \\ 0 & -0.24-0.36i & 0.41+0.34i & -0.21+0.16i & 0.30+0.30i & -0.92-0.24i \end{bmatrix} \right. \\
&\times \begin{bmatrix} 0 & 0.08-0.25i & 0.06+0.08i & -0.80+0.57i & 0.63-0.17i & 0.55+0.51i \\ 0 & 0.06-0.48i & 0.06+0.16i & 0.35+0.51i & -0.90+0.24i & -0.24+0.47i \\ 0 & -0.06+0.39i & -0.13+0.29i & 0.20+0.32i & 0.02-0.24i & -0.44+0.32i \end{bmatrix} \\
&\left. + 2 \times \begin{bmatrix} 0 & 0.6+0.1i & 0.7-0.5i & -0.5-0.3i & -0.1-0.2i & 0.4+0.1i & -0.2-0.2i & \cdots \\ 0 & 0.3-0.9i & 0.4+0.5i & 0.4+0.3i & -0.3-0.3i & -0.5+0.2i & -0.4i & \cdots \\ 0 & 0.3-0.7i & 0.1+0.2i & 0.7 & 0.1+0.2i & 0.4-0.9i & -0.4 & \cdots \end{bmatrix} \right)
\end{aligned}
\tag{2.1.1}
$$

2.1.1 Test Description

Two input signals are generated with harmonic components up to the 5th order. These signals are multiplied, and an additional independent 10th-order component is included, as defined in equation (2.1.1). The same setup is implemented using both the `DQsym` library and a reference EMT model in MATLAB/Simulink.

This example serves as a minimal working case and provides a direct comparison between dynamic phasor-based modelling and time-domain simulation.

2.1.2 Model Implementation

The EMT reference model is constructed using standard multiplication and summation blocks from the Simulink Library Browser. The `DQsym` model is built using the multiplication block

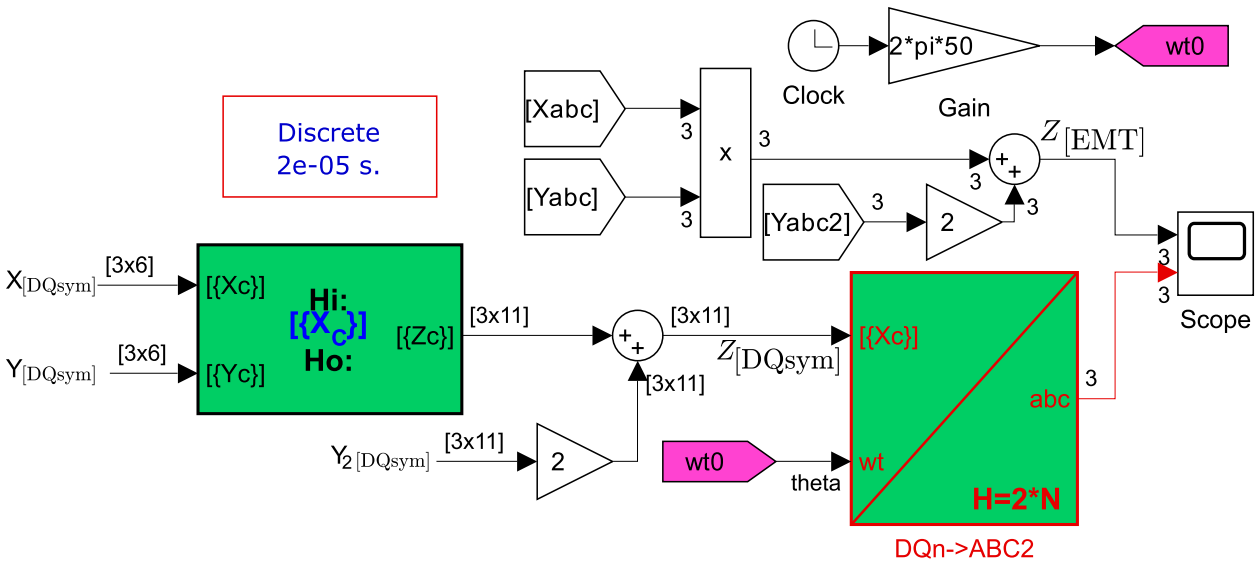


Figure 2.1.1: Multiplication example implemented using both SPS and `DQsym` blocks.

and the DQn-to-ABC transformation block.

The blocks are connected as shown in Fig. 2.1.1. The harmonic order must be specified for both blocks. The multiplication block is configured with the highest harmonic order present in the inputs (5th order). The transformation block is configured with twice that order (10th order) to correctly reconstruct the output signal.

2.1.3 Simulation and Results

The simulation is executed using the standard Simulink run command. Output signals are visualised using scope blocks connected to the model output.

This example validates the numerical consistency of the multiplication and measurement blocks. It also demonstrates the ability of the `DQsym` framework to accurately represent nonlinear harmonic interactions within a dynamic phasor formulation.

2.2 A simple RLC circuit

This section describes a basic RLC test case implemented using the `DQsym` library. The test case is intended as an introductory example to demonstrate the modeling workflow, parameter definition, and simulation execution within the `DQsym` framework. The test case file is `DQsymRLC.slx`.

2.2.1 Basic Test Description

The RLC test case represents a simple linear electrical circuit composed of resistive, inductive, and capacitive elements. It is used to validate the correct integration of state–space models, input formatting, and signal handling in `DQsym`. The circuit topology is implemented in Simulink using blocks from the Specialized Power Systems library combined with `DQsym` components. The electrical network is interfaced with the `DQsym` State–Space block, which captures the dynamic behavior of the RLC circuit.

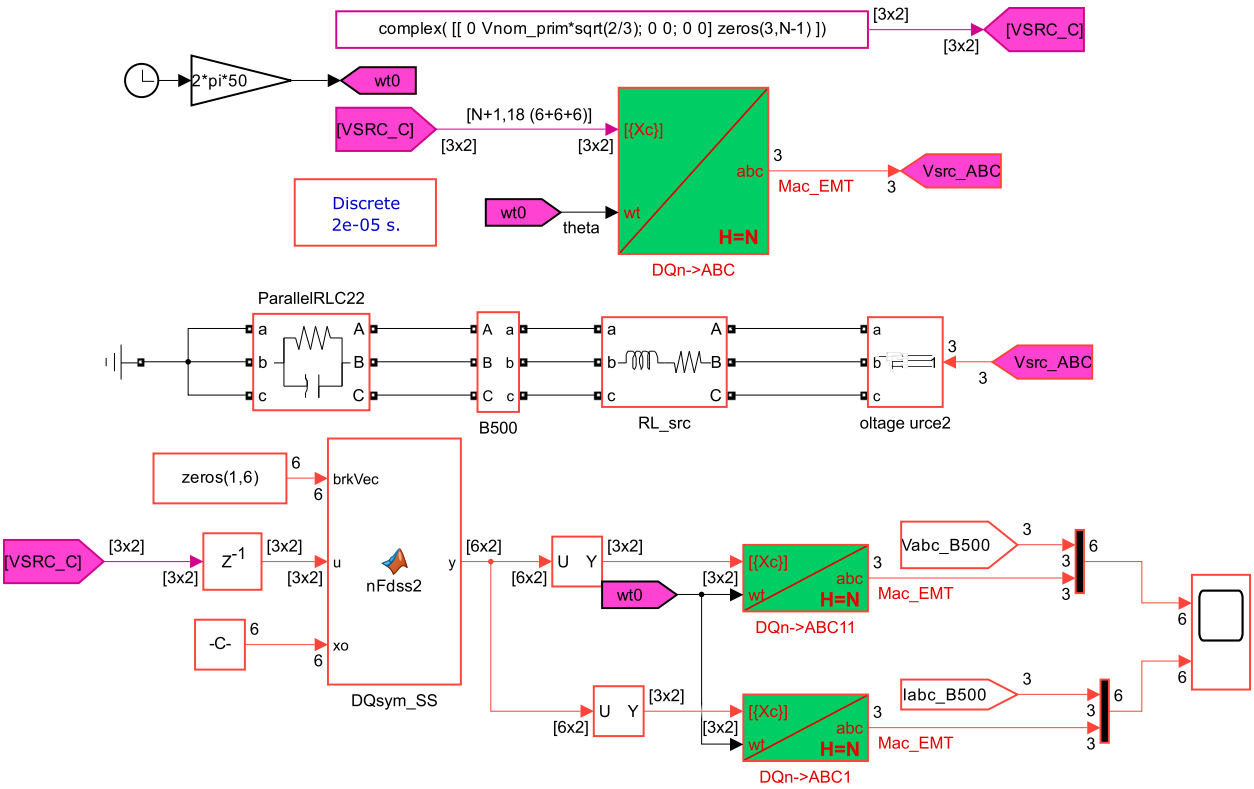


Figure 2.2.1: Overview of RLC test case in both the SPS and DQsym library blocks.

2.2.2 Parameters

All electrical parameters, including resistance, inductance, and capacitance values, are defined explicitly for the test case in `Srlc.mat`. These parameters are selected to ensure stable operation and clear observation of the system dynamics.

2.2.3 MATLAB Structure

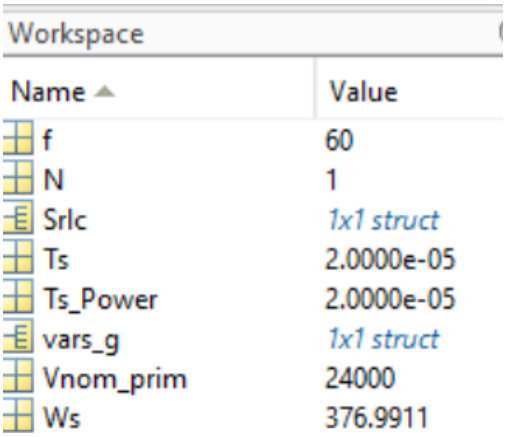
The parameters and state–space matrices of the RLC circuit are organized in a MATLAB structure `Srlc.mat`. This structure contains the system matrices, initial conditions, and any additional configuration data required by the simulation.

A dedicated MATLAB `RLC.m` file serves as the main entry point for the test case. This script loads the parameter structure, initializes the simulation environment, and prepares the corresponding Simulink model for execution.

Running the main MATLAB file (`RLC.m`) automatically compiles the test case by loading the Simulink model, assigning parameters to the `DQsym` blocks, and configuring simulation settings.

2.2.4 Run the Test Case and Results

Once complied, the test case is executed directly from the MATLAB script `RLC.m` with the following commands run: run('DQsymRLC') and sim('DQsymRLC'). Simulation results such as voltages, currents are generated and can be visualized using scopes or post-processing scripts to verify the expected dynamic response of the RLC circuit.



Workspace

Name	Value
f	60
N	1
Srlc	*1x1 struct*
Ts	2.0000e-05
Ts_Power	2.0000e-05
vars_g	*1x1 struct*
Vnom_prim	24000
Ws	376.9911

Figure 2.2.2: Test case parameters as in the workspace.

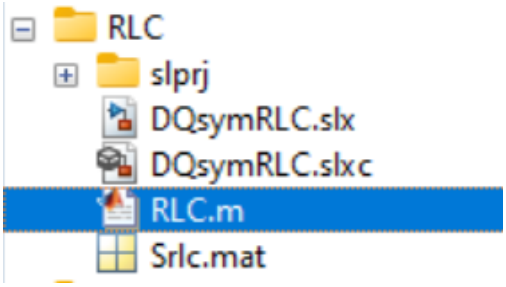


Figure 2.2.3: Test case folder tree.

2.3 MMC Station Connected to an AC Grid

This section describes a test case of a Modular Multilevel Converter (MMC) station connected to an AC grid through a transformer. The test case demonstrates full control capability of the MMC to independently regulate active and reactive power using the `DQsym` library.

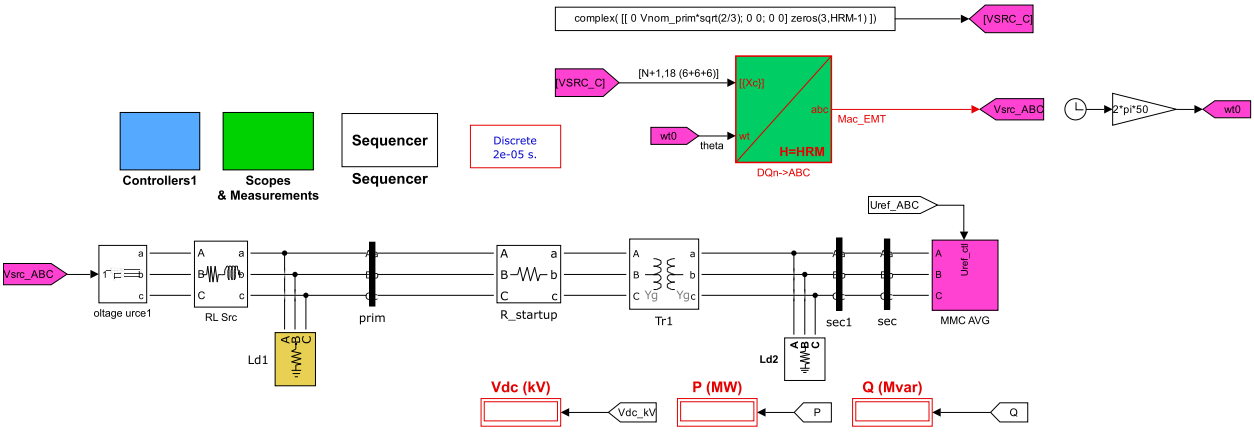


Figure 2.3.1: Overview of MMC station test case in both the SPS and DQsym library blocks.

2.3.1 Basic Test Description

The test case represents an MMC-based converter station interfacing an AC grid via a coupling transformer. The objective of the test is to validate the dynamic performance of the MMC model and its control system when regulating active and reactive power under grid-connected operation.

The system topology is implemented in Simulink using components from the Specialized Power Systems library and the `DQsym` library. It includes the MMC station, the grid-side transformer, and an equivalent AC grid model. The converter dynamics are represented using `DQsym` State–Space blocks, while the electrical interface and measurements are handled through dedicated `DQsym` input and output blocks.

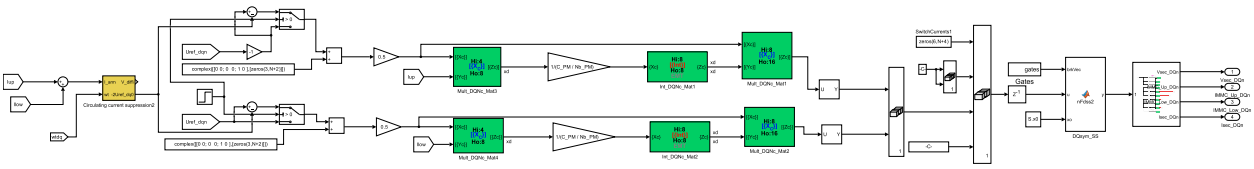


Figure 2.3.2: Upper and lower voltages as calculated using DQsym library blocks.

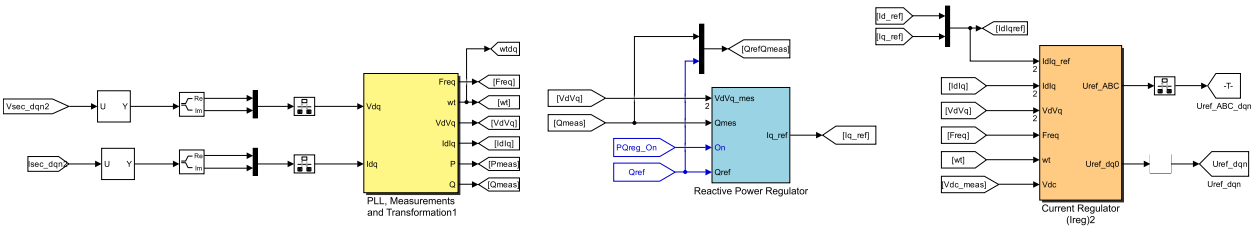


Figure 2.3.3: MMC single-station control structure.

2.3.2 Parameters

The test case parameters include MMC electrical characteristics (arm inductance, resistance, capacitance, and number of submodules), transformer parameters, and AC grid parameters such as grid voltage, frequency, and short-circuit strength. Control parameters for the inner current loops and outer active and reactive power control loops are also defined in `dqn_C_PQ_HVsMC.m` file and stored in `Sdq.mat`

2.3.3 MATLAB Structure

All system and control parameters, including the state–space matrices of the MMC model, are organized in a MATLAB structure `Sdq.mat`. This structure contains converter parameters, grid and transformer data, controller gains, and initial operating points required for simulation.

A MATLAB `dqn_C_PQ_HVsMC.m` file serves as the main execution script for the test case. This file loads the parameter structure, initializes the MMC, and configures the control references for active and reactive power before launching the Simulink model.

Executing the main MATLAB file `dqn_C_PQ_HVsMC.m` compiles the test case by loading the Simulink model, assigning parameters to the `DQsym` State–Space and control blocks, and configuring the simulation settings for grid-connected operation.

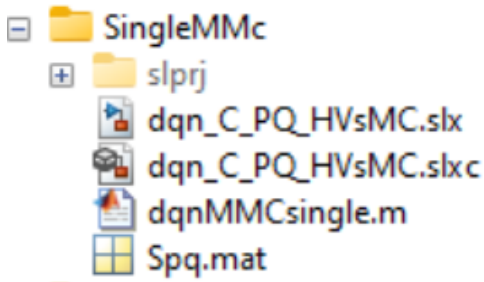


Figure 2.3.4: MMC single-station folder structure.

2.3.4 Run the Test Case and Results

After building the model `dqn_C_PQ_HVsMC.slx`, the simulation is executed from the main script using codes 1- open `dqn_C_PQ_HVsMC.slx` 2- sim `dqn_C_PQ_HVsMC.slx`. The results include AC-side voltages and currents, active and reactive power responses. These outputs are used to verify correct power control performance, steady-state tracking, and dynamic response of the MMC station connected to the AC grid.

2.4 Point-to-Point HVDC System with MMC

This section describes a point-to-point HVDC transmission system based on Modular Multilevel Converters (MMCs). The test case demonstrates the dynamic behavior and coordinated control of two MMC stations connected through a DC link using the `DQsym` library.

2.4.1 Basic Test Description

The system topology is implemented in Simulink using the Specialized Power Systems library and `DQsym` components. Each MMC station is connected to its respective AC grid through a

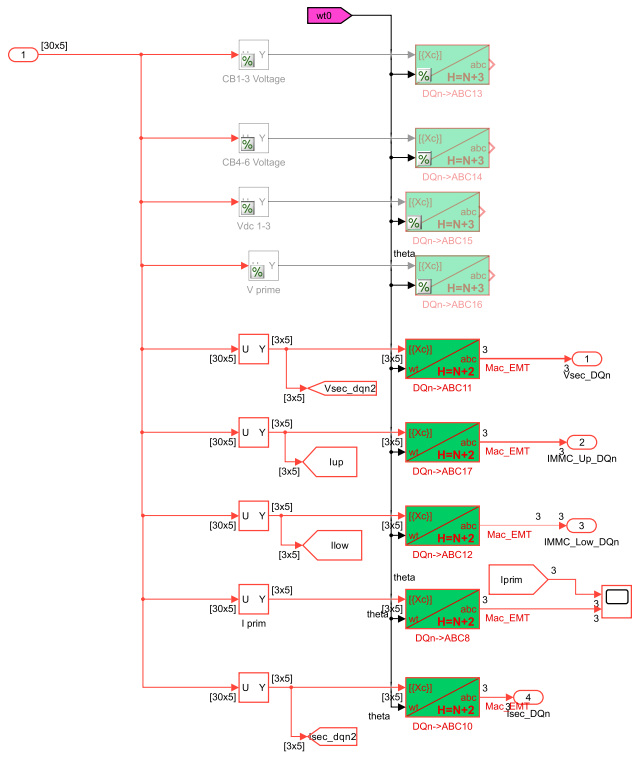


Figure 2.3.5: outputs transformed into three phase values to be compared with Matlab/Simulink model.

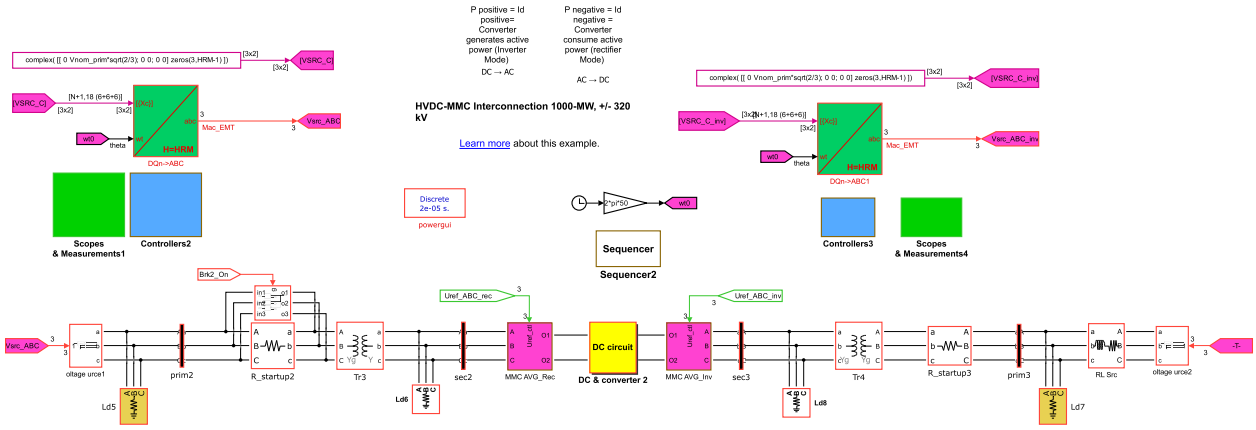


Figure 2.4.1: Overview of P2P test case simulation using *DQsym* library in Matlab/Simulink.

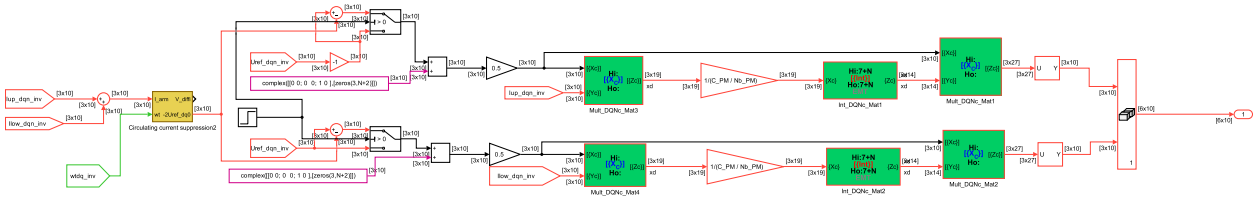


Figure 2.4.2: Overview of upper and lower arm current of the inverter MMc using *DQsym* library in Matlab/Simulink.

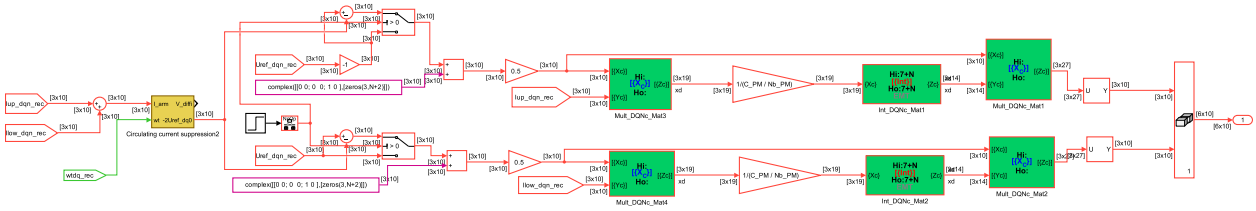


Figure 2.4.3: Overview of upper and lower arm current of the rectifier MMc using *DQsym* library in Matlab/Simulink.

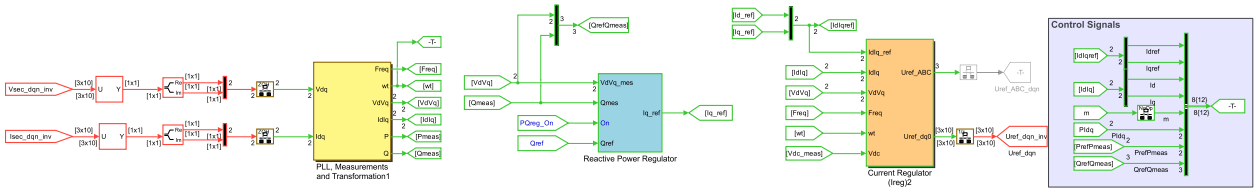


Figure 2.4.4: Overview of the inverter MMc control system using *DQsym* library in Matlab/Simulink.

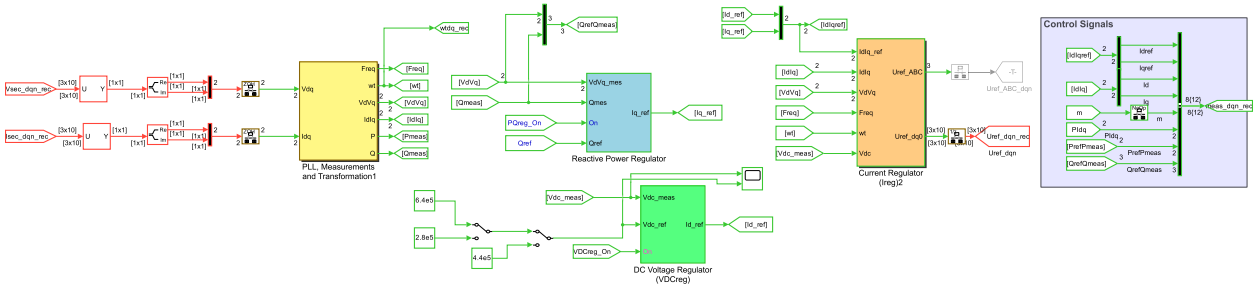


Figure 2.4.5: Overview of the rectifier MMc control system using *DQsym* library in Matlab/Simulink.

transformer. The two converters are interconnected via a DC transmission line or DC equivalent circuit. The converter dynamics are represented using `DQsym` State–Space blocks, with dedicated blocks for DQ-domain signal conversion and interfacing.

2.4.2 Parameters

The test case parameters are loaded using file `P2PHVDCMMC.m`. include MMC electrical parameters for both terminals (arm inductances, resistances, capacitances, number of submodules), DC link parameters (DC voltage rating, line resistance and inductance), transformer parameters, and AC grid characteristics at each end are all saved in `SFu.mat`. Control parameters for inner current controllers, DC voltage control, active power control, and reactive power control are also defined.

2.4.3 MATLAB Structure

All system, converter, and control parameters are organized in a structured MATLAB data object. This structure includes the state–space matrices for each MMC station, DC network parameters, controller gains, reference values, and initial operating conditions for both terminals.

A dedicated MATLAB `P2PHVDCMMC.m` file serves as the main execution script for the point-to-point HVDC test case. The script loads the parameter structures SFu.mat, assigns operating modes to each MMC station (e.g., DC voltage control or active power control), initializes references, and complies the Simulink model for simulation.

Executing the main MATLAB file complies the test case by loading the Simulink model, configuring the `DQsym` State–Space and control blocks for both MMC stations, and initializing the DC link and grid interface components.

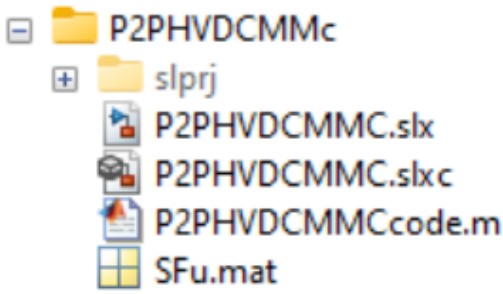


Figure 2.4.6: Test case folder structure.

2.4.4 Run the Test Case and Results

After compiling the model, the simulation is executed from the main script used codes 1- open P2PHVDCMMC.slx 2- sim P2PHVDCMMC.slx . The results include DC voltage and current waveforms, active and reactive power exchange at both AC terminals, and internal MMC state variables. These results are used to assess power flow control, DC voltage regulation, and the dynamic interaction between the two MMC stations in the HVDC system.

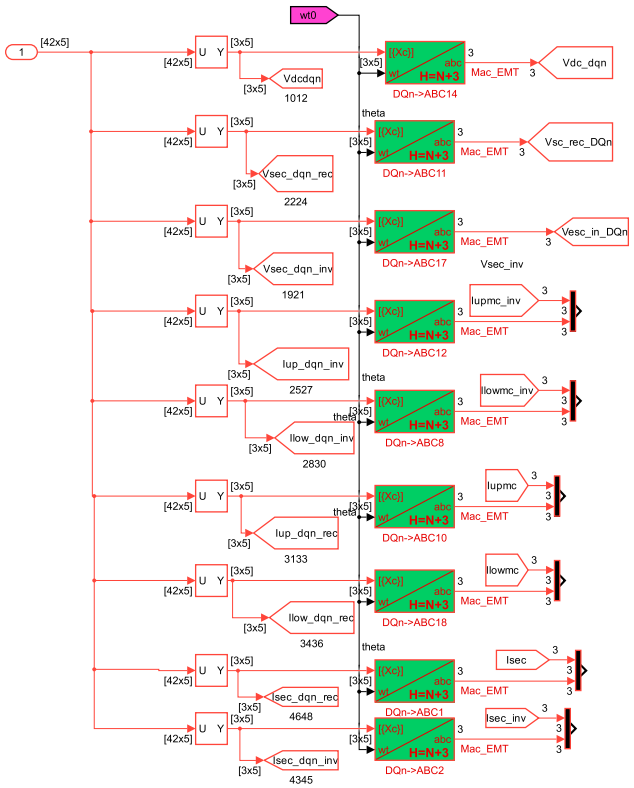


Figure 2.4.7: outputs transformed into three phase values to be compared with Matlab/Simulink model.